



Introduction To Programming Using C++ Level 2 Bitwise AND Operator



Introduction To Programming Using C++

Level 2

Bitwise AND Operator

Definition

- ◆ The Bitwise AND Operator `&` compares two numbers bit by bit.
- ◆ It returns `1` only when both corresponding bits are `1`.
- ◆ If either bit is `0`, the result for that position is `0`.

Explanation

-  Computers represent integer values using Binary numbers made of `0` and `1`.
-  The Bitwise AND Operator compares the corresponding bits of two numbers.
-  Each bit is processed independently.
-  The result contains `1` only in positions where both input numbers contain `1`.

Procedures

1. Convert the numbers to Binary.
2. Write the Binary values one below the other.
3. Compare the corresponding bits from left to right.
4. If both bits are `1`, write `1`.
5. If either bit is `0`, write `0`.
6. Convert the resulting Binary number back to Decimal if needed.

💡 Example

- ◆ Suppose we have:

```
int Result = 25 & 12;
```

```
cout << Result;
```

- ◆ In Binary:

```
25 = 00011001
```

```
12 = 00001100
```

```
-----
```

```
00001000
```

- ◆ The corresponding bits are compared:

```
00011001
```

```
& 00001100
```

```
-----
```

```
00001000
```

- ◆ `00001000` in Decimal is `8`.

➡ Result

The result is:

8

Note

- ◆ The Bitwise AND Operator works directly on the individual bits of integer values.
- ◆ The operator used for Bitwise AND is:
`&`

⊘ Warning

- ◆ Bitwise AND `&` is different from Logical AND `&&`.
- ◆ `&` works on individual bits, while `&&` is used for logical conditions.



Comparison

▨ Bitwise AND `&`

- ✓ Compares individual bits.
- ✓ Returns `1` when both corresponding bits are `1`.
- ✓ Commonly used for bit manipulation and bit masks.

▤ Logical AND `&&`

- ✓ Works with logical conditions.
- ✓ Evaluates whether two conditions are both True.
- ✓ Commonly used in `if` statements and other logical expressions.

✦✦ Summary

- ✓ Bitwise AND `&` compares numbers bit by bit.
- ✓ A bit becomes `1` only when both corresponding bits are `1`.
- ✓ `25 & 12` produces `8`.

Thank you 😊